

# SecureLab: Web Security Attack Simulations

SQL Injection · Cross-Site Scripting · Parameter Tampering

Rithanyaa M E | 71762231042 | M.Sc. Software Systems (9th Sem)

Coimbatore Institute of Technology

August 2026

# What is SecureLab?

**SecureLab** is an interactive web security simulation platform built with Python Flask and SQLite.

It demonstrates **3 real-world web application attacks** — each with a live **Vulnerable Mode** and a **Secure Mode**:

| # | Attack                     | OWASP Rank |
|---|----------------------------|------------|
| 1 | SQL Injection              | A03:2021   |
| 2 | Cross-Site Scripting (XSS) | A03:2021   |
| 3 | Parameter Tampering        | A04:2021   |

*Every attack shows exactly **what goes wrong**, **how it is exploited**, and **what one change fixes it**.*

# Why These Attacks Matter

Web applications power banking, e-commerce, healthcare, and social media.

**Poorly written code exposes users to attackers who exploit subtle flaws in how input is handled.**

- **OWASP Top 10** is the industry-standard list of the most critical web security risks
- The 3 attacks in this project appear in the **top 4** of that list
- Real breaches caused by these vulnerabilities have affected **hundreds of millions** of people

**The golden rule of web security:**  
*“Never trust user input.”*

## Attack 1 — What is SQL Injection?

SQL Injection occurs when an attacker **inserts malicious database commands** into an input field, tricking the server into running unintended operations.

### Real-world breaches:

- **Heartland Payment Systems (2008)** — 134 million credit card records stolen
- **Sony Pictures (2011)** — 1 million user accounts exposed

### What an attacker can do:

- Log into any account **without knowing the password**
- Read private data from the entire database
- Delete or corrupt database tables
- Escalate privileges to administrator

The application has a login page for a fake bank called **FakeBank**.

**What the developer intended:** The user types a username and password; the server checks if both match.

**What the attacker does:** Instead of a normal username, the attacker types a specially crafted input that **closes the username check early and comments out the password check entirely** — so the server grants access without ever verifying the password.

**Vulnerable Mode result:** Attacker logs in as Admin and sees the account balance of **Rs. 98,500** — with zero knowledge of the password.

**Secure Mode result:** The same input is treated as plain text. No match is found. Login fails.

**Root cause:** The server builds its database query by directly joining the user's input into the command string. The attacker's input becomes part of the command itself.

**The fix:** Use **parameterized queries** — the database receives the query structure and the user's value separately, so the value can never be interpreted as a command.

|                          | Vulnerable                                     | Secure                                   |
|--------------------------|------------------------------------------------|------------------------------------------|
| How query is built       | User input pasted directly into command.       | User input passed separately as a value. |
| Effect of attack payload | Password check skipped — admin access granted. | Input treated as a string — login fails. |

## Attack 2 — What is Cross-Site Scripting (XSS)?

XSS occurs when an attacker **injects malicious JavaScript** into a web page. Every user who loads that page has the script execute silently in their browser.

### Types of XSS:

- **Stored XSS** (*what this project demonstrates*) — the script is saved to the database and runs for every visitor
- **Reflected XSS** — the script travels in a URL and runs when the link is opened
- **DOM-based XSS** — the script manipulates the page entirely on the client side

### Real-world breaches:

- **Samy Worm (MySpace, 2005)** — 1 million profiles infected in 20 hours
- **British Airways (2018)** — 380,000 customers' payment card data stolen via an injected script

## XSS — How the Attack Works

The application has a public **comment board** where users can post messages visible to all visitors.

**What the developer intended:** Users post text comments; the board displays them.

**What the attacker does:** Instead of a comment, the attacker posts a piece of JavaScript code. The server saves it to the database without inspection. When any visitor loads the board, their browser encounters the stored script and **executes it as if it were part of the page**.

**Vulnerable Mode result:** An alert dialog pops up showing the visitor's cookie data — simulating a session theft attack.

**Secure Mode result:** The same payload is stored and displayed, but as **visible text only** — no execution occurs.

**Root cause:** The server stores raw user input without sanitisation, and the browser renders it as live HTML — which includes executing any scripts embedded in it.

**The fix:** Two changes work together:

- ❶ **On the server** — escape special characters before storing. Angle brackets become harmless display characters, so the browser never treats the content as HTML.
- ❷ **In the browser** — render content as plain text, not as HTML. This prevents any injected markup from being parsed or executed.

|                | Vulnerable                    | Secure                                    |
|----------------|-------------------------------|-------------------------------------------|
| Storage        | Raw script saved as-is.       | Special characters escaped before saving. |
| Display        | Rendered as live HTML.        | Displayed as plain text.                  |
| Script payload | Executes in visitor's browser | Shown as readable text — harmless.        |

## Attack 3 — What is Parameter Tampering?

Parameter Tampering occurs when an attacker **modifies values sent from the browser to the server** — such as a product price, a user role, or a discount code — before the request reaches the server.

### How attackers do it:

- Open browser **DevTools** and edit a hidden field in the page's HTML
- Modify a URL query string directly in the address bar
- Intercept and edit the HTTP request using a proxy tool like Burp Suite

### Real-world impact:

- E-commerce platforms lose revenue when attackers buy products at tampered prices
- Privilege escalation — users promote themselves to admin by changing a role parameter
- **OWASP Ranking:** A04:2021 Insecure Design

## Parameter Tampering — How the Attack Works

The application has a fake **e-commerce shop** selling products like an iPhone 16 Pro (Rs. 1,29,999).

**What the developer intended:** The user selects a product and places an order; the server charges the correct price.

**What the attacker does:** The page includes the product price as a hidden field in the form. Using DevTools, the attacker changes the price field from Rs. 1,29,999 to **Rs. 1** before submitting the order. The server reads whatever the form sends — and trusts it.

**Vulnerable Mode result:** The order log records the iPhone as purchased for **Rs. 1**, flagged with a TAMPERED badge.

**Secure Mode result:** The server ignores the client-sent price entirely and fetches the real price from the database. The real price is always charged.

**Root cause:** The server trusts a price value that the client (browser) sent. Since the client is fully under the attacker's control, any value coming from it can be manipulated.

**The fix: Never use client-supplied values for business-critical data.** Instead, look up the authoritative value from the server's own database using only the product ID.

|                | Vulnerable                         | Secure                                    |
|----------------|------------------------------------|-------------------------------------------|
| Price source   | Taken from the submitted form.     | Fetches from the database by product ID.  |
| Client control | Client dictates the price charged. | Client value is completely ignored.       |
| Result         | iPhone bought for Rs. 1.           | Real price (Rs. 1,29,999) always charged. |

| Layer      | Technology         | Role in the Project                       |
|------------|--------------------|-------------------------------------------|
| Backend    | Python + Flask     | Handles all routes and attack logic.      |
| Database   | SQLite             | Stores users, comments, products, orders. |
| Frontend   | HTML5 + CSS3       | Minimal, clean light-mode interface.      |
| Scripting  | Vanilla JavaScript | Live terminal output, mode switching.     |
| Templating | Jinja2             | Renders server-side HTML pages.           |

**Why Flask?** Flask is deliberately minimal — it exposes raw database queries and input handling without hiding them behind framework abstractions, making vulnerabilities clearly visible and demonstrable.

## Project Features

| Feature              | Description                                            |
|----------------------|--------------------------------------------------------|
| Dual mode per attack | Toggle between Vulnerable and Secure on the same page. |
| Live terminal output | Shows the exact query or server response in real time. |
| Payload chips        | One-click auto-fill of attack inputs for easy demo.    |
| Code diff panel      | Side-by-side before/after code on every attack page.   |
| Order log            | Persistent table with TAMPERED / OK badges.            |
| Comment board        | XSS scripts execute live in vulnerable mode.           |
| Reset button         | Comment board can be reset to seed data.               |
| Toast notifications  | Instant feedback on every form action.                 |

## One principle prevents all three attacks:

*"The client is always the enemy. Validate, sanitise, and verify everything on the server."*

| Attack              | What Goes Wrong                                     | The Fix                                            |
|---------------------|-----------------------------------------------------|----------------------------------------------------|
| SQL Injection       | User input becomes part of a database command.      | Separate the command from the data.                |
| XSS                 | User input is rendered as live HTML in the browser. | Escape output; render as plain text.               |
| Parameter Tampering | Server trusts values sent by the client.            | Always fetch authoritative values from the server. |

**Thank you!** | Rithanyaa M E | 71762231042